NATIONAL UNIVERSITY OF SINGAPORE
SCHOOL OF COMPUTING

MID-TERM QUIZ FOR
Semester 1, AY2014/2015

CS3230 – DESIGN AND ANALYSIS OF ALGORITHMS

20 September 2014 Time Allowed: 1.5 hour

Instructions to Candidates:

1. This examination paper contains **FOUR** questions and comprises **10** (**TEN)** printed pages, including this page.
2. Answer **ALL** questions.
3. Write ALL your answers in this examination book. (Use the blank pages provided if you need to.)
4. This is an **OPEN BOOK** examination.

Name: _____

QUESTION	POSSIBLE	SCORE
Q1	15	
Q2	15	
Q3	10	
Q4	10	
TOTAL	**50**	

Fun Bonus Question: (1 point)

Give the full name of the *time consuming* operation CTR_k introduced in CS3230.

_____

IMPORTANT NOTE:

- *You can **freely quote** standard algorithms and data structures covered in the lectures, textbook, and homeworks. Explain **any modifications** you make to them;*
- *Unless otherwise specified, you are expected to **prove (justify)** your results.*
- *All logarithms are base 2, unless otherwise stated.*

Q1. (15 points)

(a) **(8 Points)** Rank the following functions in *ascending order of growth*; that is, if function $g(n)$ *immediately* follows function $f(n)$ in your list, then it should be the case that $f(n)$ is $O(g(n))$.

$$g_1(n) = 2\sum_{k=1}^{n} n(\lg k) \qquad g_2(n) = 256n^2(\lg\lg n)$$

$$g_3(n) = 4\sum_{k=1}^{n} k(\lg n) \qquad g_4(n) = 2^{(3\lg n)}$$

To simplify notations, you should write $f(n) << g(n)$ to mean $f(n) = o(g(n))$, and $f(n) \equiv g(n)$ to mean $f(n) = \Theta(g(n))$.

No need to prove the results.

SOLUTION: (in increasing order of growth rate)

Q1. (continued)

(b) (4 points) Asymptotic Notations (by definition)

Let $f(n) = 8n^3 + 4n^2 - 2n + 1$

By using the definitions of Θ, prove that $f(n) = \Theta(n^3)$

Note: You cannot *just give* the constant c_1 c_2, and n_0. You need to *prove* it.

(c) (3 points) (Limit theorem & L'Hopital's rule)

Given that that $h(n) = n^2$, use the limit theorem to prove that $h(n) = o(2^n)$.

[*Hint:* Note that $\frac{d}{dx}(2^x) = D * 2^x$, where $D = \log_e(2)$, is a constant.]

Q2. (15 points)

(a) **(10 points)** Solve the following recurrences. Whenever possible, give your answers in Θ–notation. For simplicity, assume that $R(n)$, $S(n)$, $U(n)$ are some small constants when $n \leq 4$.
[Show your working.]

(i) (3 points) $R(n) = 4R(n/16) + 25n^{0.5}$

(ii) (3 points) $S(n) = 9S(n/3) + 5n^{1.5}$

Q2.(a) (continued)

(iii)* (4 points) $$U(n) = \frac{2}{(n-1)} \sum_{k=1}^{n-1} U(k) + 3n$$

(*Hint:* Reuse the appropriate "method".)

(use blank page 11 if you need more space)

Q2. (continued)

(b) **(5 points) [Radix Sort]**
Run RADIX-SORT on the following list of English words shown below.
Starting with the input in the leftmost column, show the list after each successive sorting step.

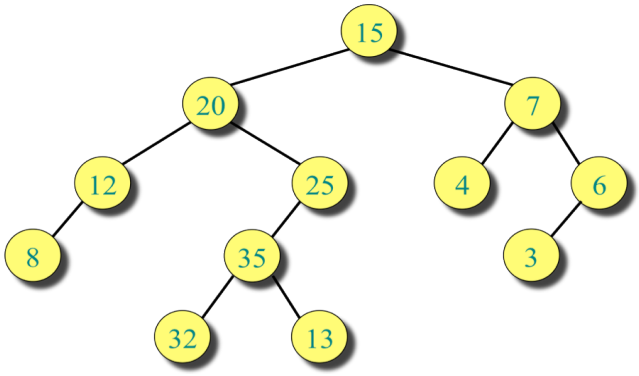
BIG	_____	_____	_____
SPY	_____	_____	_____
HOT	_____	_____	_____
TEA	_____	_____	_____
RUG	_____	_____	_____
TAR	_____	_____	_____
POT	_____	_____	_____
TAB	_____	_____	_____
SEA	_____	_____	_____
PAR	_____	_____	_____

Use the space below for your working (if needed):

Q3. (10 points)

You are given a binary tree T with n nodes and height h. Each node v in the tree has a real-number key $k(v)$. For simplicity, you can assume that these keys are *all distinct*. Then, we say that a node v is a *local maximum* if its key is *greater than* that of *all its adjacent nodes*, i.e. its parent (if it exists) and all its children (if they exist).

(a) **(2 point)** Find a local maximum in the binary tree T given below.



Answer: Local Maximum = Node with key _____

Q3. (continued)

(b) **(8 point)** Describe an $O(h)$ algorithm for finding a *local maximum* in the tree T.

[*Note:* No need to give detailed code. Just give high-level pseudo-code.
You can also illustrate your algorithm with one or more examples.]

(use blank page 11 if you need more space)

Q4. (10 points)

[Finding *the* Missing Integer]
You are given an array A[1..n] of size n, where $n = (2^k-1)$. You are told that the *unsorted* array A[1..n] contain all the integers from 0 to n (inclusive), except *one*.

For k=3, an example is A[1..7]=[3, 0, 2, 6, 1, 4, 7]. Here the *missing number* is 5.

(a) (5 points) [Finding the Missing Integer]
Give a $\Theta(n)$ algorithm to find the missing integer.
[*Hint:* Use an additional array B[0..n] if necessary.]

Q4. (continued)

[This part is non-trivial. Do this *last* if you do not have an immediate solution.]

(b) (5 point)**

However, you are *now* told that you cannot directly access the integers in A with a single query operation. (*So, your algorithm from (a) don't work now.*)

The elements in A are represented in binary and stored in k bits, namely,
$(b_{k-1}\ b_{k-2} \ldots b_1\ b_0)$ where each b_j is a binary bit. (*Note:* $k = \lg n$)

You can make the following query operation:

BIT-QUERY(j, i) : "fetch the jth bit of $A[i]$" (constant $\Theta(1)$ time)

which returns a 0 or a 1 in constant $\Theta(1)$ time.

To illustrate this, we show an example using the array $A[1..7]$ above and give some sample queries. Remember, the array $A[1..n]$ is hidden from you.

Hidden Array:	**Sample Queries**
$A[1] =$ 0 1 1	BIT-QUERY(0,2) = 0
$A[2] =$ 0 0 0	
$A[3] =$ 0 1 0	BIT-QUERY(1,3) = 1
$A[4] =$ 1 1 0	
$A[5] =$ 0 0 1	BIT-QUERY(2,6) = 1
$A[6] =$ 1 0 0	
$A[7] =$ 1 1 1	

Using this BIT-QUERY operation, and additional storage if needed, describe an algorithm for finding the *missing integer* with $\Theta(n)$ worst-case running time. Also illustrate how your algorithm works on the above example.

[*Note:* You do need to give code or detailed pseudo-code.]
[*Note:* You can use k queries to find all the bit of $A[i]$, and hence determine the value of $A[i]$, but *this step alone* will take a total of $\Theta(nk) = \Theta(n \lg n)$ time.]

(*This page is blank. Use it for your answer, if needed.*)

-- End of Paper --